

选择题（每题 2 分）

采用分治法解决问题时，算法时间复杂度分析典型的递推方程为 $T(n) = \begin{cases} O(1) & n = 1 \\ aT\left(\frac{n}{b}\right) + f(n) & n > 1 \end{cases}$

对于某一个问题，若 $f(n) = O(n)$ ，且 $a > b > 0$ ，则求该问题的时间复杂度为（B）

A $O(n \log n)$ B $O(n^{\log_b a})$ C $O(n)$ D $O(n^2 \log n)$

$$T(n) = \begin{cases} O(1) & n = 1 \\ aT(n-1) + f(n) & n > 1 \end{cases} \rightarrow T(n) = a^{n-1}T(1) + \sum_{i=0}^{n-2} a^i f(n-i)$$

$$T(n) = \begin{cases} O(1) & n = 1 \\ aT\left(\frac{n}{b}\right) + f(n) & n > 1 \end{cases} \rightarrow T(n) = n^{\log_b a} T(1) + \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right)$$

$$\Rightarrow f(n) = c \rightarrow T(n) = \begin{cases} O(\log n) & a = 1 \\ O(n^{\log_b a}) & a \neq 1 \end{cases}$$

$$\Rightarrow f(n) = cn \rightarrow T(n) = \begin{cases} O(n) & a < b \\ O(n \log n) & a = b \\ O(n^{\log_b a}) & a > b \end{cases}$$

（多选）以下哪个问题适合用贪心算法求解（BCD）

A 0-1 背包问题 B 哈夫曼编码 C 单源最短路径 D 最优装载问题

贪心算法之所以不能求解 0-1 背包问题，是因为在贪心选择的情况下，算法无法保证背包能被装满，导致有部分背包空间浪费了。

适合用回溯法和分支定界法求解的问题均需要满足（C）

A 重叠子问题 B 最优子结构性质 C 多米诺性质 D 独立子问题

重叠子问题：动态规划

最优子结构：贪心、动态规划

多米诺性质：回溯、分支定界

独立子问题：分治

贪心选择性质：贪心

以下哪个问题的解空间树是排列树（C）

A n 皇后问题 B 最大团问题 C 批作业调度问题 D 图的 m 着色问题

排列树：TSP 问题、批作业调度问题、电路板排列问题

子集树：n 皇后问题、装载问题、0-1 背包问题、最大团问题、图的 m 着色问题、高精度数问题、布线问题

以下关于渐进符号性质表达正确的是（C）

$$A \ O(f(n) + g(n)) = O(\min\{f(n), g(n)\})$$

$$B \ f(n) = O(g(n)), g(n) = O(h(n)) \rightarrow h(n) = O(f(n))$$

$$C \ f(n) = \Theta(g(n)), g(n) = \Theta(h(n)) \rightarrow f(n) = \Theta(h(n))$$

$$D \ f(n) = O(g(n)) \Leftrightarrow g(n) = O(f(n))$$

$$\text{如果 } f(n) = O(g(n)), g(n) = O(h(n)) \rightarrow f(n) = O(h(n))$$

$$\text{如果 } f(n) = \Omega(g(n)), g(n) = \Omega(h(n)) \rightarrow f(n) = \Omega(h(n))$$

如果 $f(n) = \Theta(g(n)), g(n) = \Theta(h(n)) \rightarrow f(n) = \Theta(h(n))$
 $O(f(n)) + O(g(n)) = O(f(n) + g(n)) = O(\max\{f(n), g(n)\})$
 $O(f(n)) * O(g(n)) = O(f(n) * g(n))$

回溯法一般是按 (A) 策略搜索问题的解空间树

A 深度优先 B 广度优先 C 广度深度结合 D 活结点优先

回溯法: 深度优先

分支定界法: 广度优先或函数优先

适合应用贪心算法与动态规划算法求解的问题所具有的共同特点是 (B)

A 重叠子问题 B 最优子结构性质 C 贪心选择性质 D 独立子问题

对于物品数量为 n , 背包容量为 c 的情况下的 0-1 背包问题, 若用回溯法求解, 算法的时间复杂度为 (D)

A nc B 2^{nc} C $c2^n$ D $n2^n$

回溯法: $O(f) = \text{遍历树的时间复杂度} * \text{计算约束的时间 (生成一个节点的时间)}$, 子集树遍历 $O(2^n)$, 排列树遍历 $O(n!)$

装载问题: 子集树, $O(2^n)$

批处理作业调度: 排列树, $O(n!)$

N 皇后问题: 子集树, $O(n!)$

0-1 背包: 子集树, $O(n2^n)$

最大团问题: 子集树, $O(n2^n)$

图的 m 着色问题: 子集树, $O(nm^n)$

TSP 问题: 排列树, $O(n!)$

分支限界法:

装载问题: $O(2^n)$

0-1 背包问题: $O(2^n)$

TSP 问题: $O(n!)$

布线问题: $O(mn)$

最大团问题: $O(n2^n)$

批作业调度问题: $O(n!)$

分治法:

全排列: $O(n!) \rightarrow T(n) = nT(n-1) + O(n)$

整数划分

二分搜索: $O(\log n) \rightarrow T(n) = T\left(\frac{n}{2}\right) + O(1)$

大整数乘法: $O(n^{\log 3}) \rightarrow T(n) = 3T\left(\frac{n}{2}\right) + O(n)$

矩阵乘法: $O(n^{\log 7}) \rightarrow T(n) = 7T\left(\frac{n}{2}\right) + O(n^2)$

合并排序: $O(n \log n) \rightarrow T(n) = 2T\left(\frac{n}{2}\right) + O(n)$

快速排序：最坏： $O(n^2) \rightarrow T(n) = T(n-1) + O(n)$ 、最好和平均： $O(n \log n) \rightarrow T(n) = 2T\left(\frac{n}{2}\right) + O(n)$

线性时间选择： $O(n) \rightarrow T(n) \leq \begin{cases} C_1 & n < 75 \\ C_2 n + T\left(\frac{n}{5}\right) + T\left(\frac{3n}{4}\right) & n \geq 75 \end{cases}$

最近点对问题： $O(n \log n) \rightarrow T(n) = 2T\left(\frac{n}{2}\right) + O(n)$

棋盘覆盖问题： $O(4^n) \rightarrow T(n) = 4T(n-1) + O(1)$

动态规划：

矩阵连乘问题： $O(n^3) \rightarrow m[i][j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i][k] + m[k+1][j] + p_{i-1}p_kp_j\} & i < j \end{cases}$

凸多边形最优三角剖分： $O(n^3) \rightarrow t[i][j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{t[i][k] + t[k+1][j] + w(v_{i-1}v_kv_j)\} & i < j \end{cases}$

最长公共子序列：计算 $O(mn)$ ，构造 $O(m+n) \rightarrow c[i][j] = \begin{cases} 0 & i > 0; j = 0 \\ c[i-1][j-1] + 1 & i, j > 0; x_i = y_i \\ \max\{c[i][j-1], c[i-1][j]\} & i, j > 0; x_i \neq y_i \end{cases}$

电路布线问题： $O(n^2)$

图像压缩问题： $O(n) \rightarrow s[i] = \min_{1 \leq k \leq \min\{i, 256\}} \{s[i-k] + k * bmax(i-k+1, i)\} +$

11, $bmax(i, j) = \left\lceil \log(\max_{1 \leq k \leq j} \{p_k\} + 1) \right\rceil$

最大子段和问题： $O(n) \rightarrow b[j] = \max\{b[j-1] + a[j], a[j]\} \quad 1 \leq j \leq n$

投资问题：

0-1 背包问题： $m(i, j) = \begin{cases} \max\{m(i+1, j), m(i+1, j-w_i) + v_i\} & j \geq w_i \\ m(i+1, j) & 0 \leq j < w_i \end{cases}$

$m(n, j) = \begin{cases} v_n & j \geq w_n \\ 0 & 0 \leq j < w_n \end{cases}$

0-N 背包问题： $m(i, j) = \begin{cases} \max\{m(i-1, j), m(i, j-w_i) + v_i\} & j \geq w_i \\ m(i-1, j) & 0 \leq j < w_i \end{cases}$

$\begin{cases} m(0, j) = 0 \\ m(i, 0) = 0 \end{cases}$

最优二叉搜索树： $O(n^3)$

流水作业调度： $O(n \log n)$

序列匹配： $O(mn)$

贪心：

活动安排：早完成的活动先安排， $O(n \log n)$

背包问题和 0-1 背包问题：背包问题可用贪心，0-1 背包不行，单位重量价值最高的物品先装， $n \log n$

最优装载问题：集装箱按从轻到重排序，轻者先装， $O(n \log n)$

最优前缀码问题（哈夫曼）： $O(n \log n)$

最小生成树问题（Prim（加点）和 Kruskal（加边））：

当图的节点数为 n 时，Prim 算法所需的计算时间是 $O(n^2)$

当图的边数为 e 时，Kruskal 算法所需的计算时间是 $O(e \log e)$ 。

当 $e = \Omega(n^2)$ 时，Kruskal 算法比 Prim 算法差，但当 $e = o(n^2)$ 时，Kruskal 算法却比 Prim 算法好得多。

单源最短路径问题（Dijkstra 算法）： $O(n^2)$

多机调度问题： $O(n \log n)$

硬币找零问题：只有当 $2 * v[i] \leq v[i + 1]$ ，找零问题才满足贪心性质

填空题（每空 2 分）

算法是由若干指令组成的无穷序列，需要满足 输入、输出、确定性、有效性、有限性

算法的效率一般可用从两个方面衡量，分别称为 时间复杂度 和 空间复杂度。

假设算法 A 理论上的时间复杂度 $T(n) = 2^n$ ，现在有两台同类型计算机 M1 和 M2。M2 的计算速度是 M1 的 64 倍。若在 M1 和 M2 上分别测试算法 A，则在相同的时间内，M1 和 M2 能够求解的问题规模 n_1 和 n_2 的关系为 $n_1 = n_2 - 6$ 。若算法 B 理论上的时间复杂度为 $T(n) = n^2$ ，若在 M1 和 M2 上分别测试算法 B，则 M1 和 M2 求解相同规模问题所耗费的时间 t_1 和 t_2 的关系为 $t_1 = 64 * t_2$ 。

$$A: t = 2^{n_1} = \frac{2^{n_2}}{64} \rightarrow n_1 = n_2 - 6$$

$$B: n_1 = \sqrt{t_1} = \sqrt{64 t_2} \rightarrow t_1 = 64 * t_2$$

用回溯法求解问题时，可将问题解空间看作一棵树状结构，根据问题不同，解空间树一般可分为子集树和排列树两种结构，对于 N 皇后问题，其解空间树是 子集树 结构。对于 TSP 问题，其解空间树是 排列树 结构。

用分治法求解 n 个元素的全排列问题时，算法时间复杂度递推方程为 $T(n) \equiv \begin{cases} O(1) & n = 1 \\ nT(n-1) + O(n) & n \geq 1 \end{cases}$ 。

0-1 背包问题可用适用动态规划、回溯和分支定界三种方法求解，其中排序对求解过程没有影响的方法是 动态规划。

若序列 $X = \{A, B, C, B, D, A, B\}$, $Y = \{B, D, C, A, B, A\}$ ，则 X 和 Y 的一个最长公共子序列为 $\{B, D, B, A\}$ ；采用动态规划方法求解该类问题（序列 X 和 Y 的规模分别为 m 和 n ）的时间复杂度为 $O(mn)$ 。

采用分治法解决问题时，算法时间复杂度分析典型的递推方程为 $T(n) = \begin{cases} O(1) & n = 1 \\ aT\left(\frac{n}{b}\right) + f(n) & n > 1 \end{cases}$ ，

对于某一问题，若 $f(n) = O(1)$ 且 $a = 1$ ，则求解该问题的时间复杂度为 $O(\log n)$ ；若 $f(n) = O(n)$ 且 $a = b > 1$ ，则求解该问题的时间复杂度为 $O(n \log n)$ ；若 $f(n) = O(n)$ 且 $a = b^2 > 1$ ，则求解该问题的时间复杂度为 $O(n^2)$ ；若 $f(n) = O(n)$ 且 $a, b > 1$ ，若希望将算法时间复杂度降

低为不超过线性情况，则 a 和 b 应该满足的条件为 $1 < a < b$ 。

应用贪心算法求解问题的关键在于 贪心策略的选择。

动态规划算法的求解步骤有四步，分别为找出最优子结构、建立递推关系式、自底向上计算最优解、构造最优解。

分治的求解步骤有三步：划分、求解子问题、合并

回溯和分支定界的求解步骤：定义解空间（问题解向量、解向量分量取值集合、构造解空间树）、判断问题是否满足多米诺性质、确定界函数和剪枝函数，搜索解空间树（、确定存储搜索路径的数据结构）

对于有 n 个顶点， m 条边的连通带权图，如果用 Kruskal 算法求其最小生成树，则时间复杂度为 $O(m \log m)$ ；如果用 Prim 算法求其最小生成树，则时间复杂度为 $O(n^2)$ 。

遗传算法主要借鉴的是生物界的自然选择和自然遗传，遗传操作有选择、交叉和变异。

遗传算法评价一个解的好坏用适应度函数，其设计标准是适应度函数值越大，解的质量越好。

遗传算法与收敛性相关的因素：种群规模、选择操作、交叉概率、变异概率。

基本遗传算法主要由 4 部分组成分别是编码、适应度函数、遗传算子、运行参数。

分支定界法主要有两种搜索方式，分别为FIFO 搜索和优先队列搜索。

节点 v 的上界 $UB(v)$ 是指从 v 出发得到的所有叶子节点的效益值均不大于 $UB(v)$ ，如果所有叶子节点的最小效益值等于节点 v 的下界 $LB(v)$ ，则 $LB(v)$ 为节点 v 的下确界。

分支限界法求解极小化问题时，主要用节点的下界进行剪枝。

回溯法一般按照深度优先的策略搜索解空间树，在搜索的过程中主要通过约束函数和界限函数避免无效的搜索。

算法设计与分析题（每题 10 分）

设 B 为数组， $B[1 \cdots n]$ 中存储了 n 个元素，分析下面的算法并回答问题。

```
ALG( $B, n$ )
  IF  $n = 1$  THEN RETURN  $B[1]$ 
  ELSE
     $temp = ALG(B, n - 1)$ 
    IF  $temp \geq B[n]$  THEN RETURN  $temp$ 
    ELSE RETURN  $B[n]$ 
```

(1) 该算法的输出是什么？

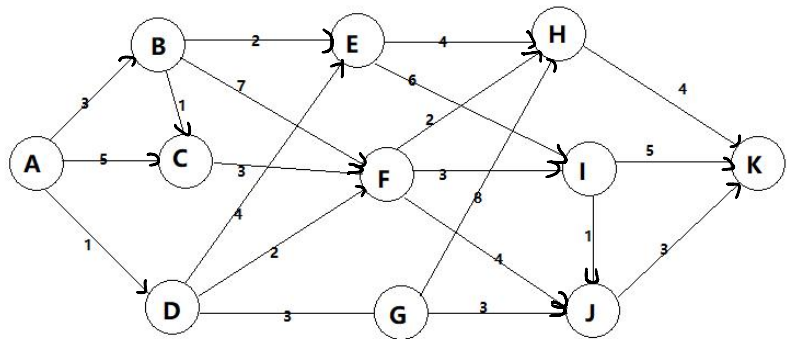
- (2) 以比较作为基本操作，给出算法 ALG 的时间复杂度递推方程 $T(n)$ ，并求解算法 ALG 的时间复杂度。
- (3) 对于求解该问题，算法 ALG 效率是否已最高，为什么？

(1) $\max_{1 \leq i \leq n} B[i]$

(2) $T(n) = T(n - 1) + O(1) \rightarrow T(n) = O(n)$

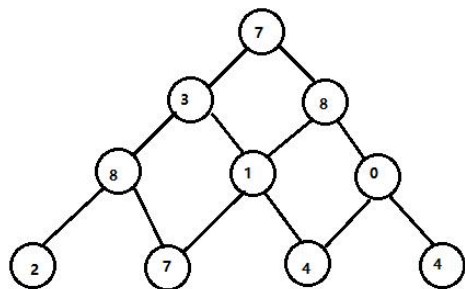
(3) 是，因为要找出数组中最大元素，需要遍历数组，效率至少是到 $O(n)$

有如下图所示的地图，图中有 A-K 共 11 个地点，两点之间的路径长度为边中的数字，如 A 到 B 的路径为 3。求 A 到 K 的最短路径？



- (1) 若用动态规划算法求解该问题，请给出最短路径的递推定义
- (2) 针对上图，按照你所给出的递推定义，逐步求解从 A 走到 K 的最短路径

有形如下的数塔，若要求从数塔顶层出发，每个节点可以选择向左走或向右走，一直走到塔底，使得走过路径上节点的数值之和最大，回答以下问题：



- (1) 若用动态规划算法求解该问题，请给出最优值的递推定义，包括初值说明
- (2) 针对上图的数塔，按照你所给出的递推定义，逐步求解从塔顶走到塔底的最大数值之和与得到最大数值和的路径

(1) 定义数塔的存储结构是 $v[n+1][n+1]$

0	0	0	0	0
0	7	3	8	2
0	0	8	1	7
0	0	0	0	4
0	0	0	0	4

DP 的初值为 $dp[n+1][n+1]$

0	0	0	0	0
0	7	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

则递推公式为 $dp[i][j] = \begin{cases} v[i][j] & i=j=1 \\ \max\{dp[i-1][j-1], dp[i-1][j]\} + v[i][j] & i \geq j > 1 \end{cases}$

$dp[i][j]$ 表示从顶层到第 j 层，走过路径上节点的最大数值之和， $v[i-1][j-1]$ 表示 $v[i][j]$ 的左父母节点， $v[i-1][j]$ 表示 $v[i][j]$ 的右父母节点。最优值为 $\max_{1 \leq j \leq n} \{dp[n][j]\}$

(2)

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	7	0	0	0	0	7	10	0	0	0	7	10	18	0	0	7	10	18	20
0	0	0	0	0	0	0	15	0	0	0	0	15	16	0	0	0	15	16	25
0	0	0	0	0	0	0	0	0	0	0	0	0	15	0	0	0	0	15	20
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	19

最优值为 $\max_{1 \leq j \leq n} \{dp[n][j]\} = dp[4][2]$ ，路径自底向上为 $25 \rightarrow (\text{左})18 \rightarrow (\text{右})10 \rightarrow (\text{右})7$ ，

自顶向下即 $7 \rightarrow (\text{左})10 \rightarrow (\text{左})18 \rightarrow (\text{右})25$

已知一个国家有 n 个城市，已知每个城市过去一年的 GDP 总量，请找出其中 GDP 总量最接近的两个城市，回答以下问题：

- (1) 用数学方法描述该问题的求解目标
- (2) 为该问题设计一个有效算法，需要给出算法思想和伪代码
- (3) 对你所设计的算法进行时间复杂度分析
- (4) 对于求解该问题，你所设计的算法的效率是否已最高，为什么？

(1) 已知 $city[1...n]$ 中存储了 n 个城市过去一年的 GDP 总量，请找出两个城市 i 和 j ，使 $|city[i] - city[j]| \leq |city[m] - city[n]|$ ，对 $\forall m, n \in 1...n$ 都成立。

(2) 算法思想：先对 $city$ 数值从大到小排序，排序的过程中，记录排序后的下标与原数组下标之间的映射关系，然后遍历数组计算 $minusGDP = |city[i'] - city[i' + 1]|$ ，找出最小的那个 GDP 差值对应的 i' 和 $i' + 1$ ，然后通过映射表，找到原本的两个城市。

伪代码：

```
void minusGDP(city[n]){
    sort(city, table); // 对 city 数组从大到小排序，并将排序前后下标的映射关系存入 table 字典
    minusGDP = city[1] - city[2] // 初值为第一个城市和第二个城市的 GDP 差值
    a = 1, b = 2; // 存储最小差值下标
    for i = 2 to n - 1 do
        tmp = city[i] - city[i + 1];
        if tmp < minusGDP then
            minusGDP = tmp;
            a = i, b = i + 1;
    end
```

```

        output(table[a], table[b]);    // 输出最小 GDP 差值对应的下标的原城市
    }

```

(3) 时间复杂度: $T(n) = O(n \log n) + O(n) = O(n \log n)$

(4) $O(n \log n)$ 的效率已经是最高, 这个问题可以类比书上的最接近点对问题的一维情况, 书上证明了 $O(n \log n)$ 的效率是该类问题的下界。

已知有 n 个学生, 若要求找出其中体重最为接近的两位学生, 回答以下问题:

- (1) 用数学方法描述该问题求解目标
- (2) 为该问题设计一个有效算法, 需要给出算法思想和伪代码
- (3) 对你所设计的算法进行时间复杂度分析

同上

算法设计与实现 (每题分值不定, 大概 10+)

v

要求:

- (1) 分析该问题适合用何种算法求解
- (2) 给出问题解的形式
- (3) 给出求解问题的代码框架
- (4) 分析算法的时间复杂度
- (1) 适合用蛮力法, 回溯法, 分支限界法求解。
- (2) 问题解的形式: n 元组 $a = \langle a_1, a_2, a_3, \dots, a_n \rangle$, 其中 a_i 表示将第 a_i 个工作分配给第 i 个人。
- (3)

使用回溯法, 递归框架:

```

bestCost; // 当前最小费用
besta[n]; // 当前最小费用对应的解
cc = 0;    // 存储当前的费用
void backtrack(int t){
    if(t > n) output(a);
    else
        for(int i = f(n, t); i ≤ g(n, t); i++){
            a[t] = h(i);
            if(constraint(t) && bound(t))
                backtrack(t + 1);
        }
}
}
output(a){
    if(cost(a) < bestCost){
        bestCost = cost(a);
        besta = a;
    }
}
}
f(n, t) = t, g(n, t) = n

```

```

constraint(t):
check(t,j){ // 第 t 个工人是否能安排做第 j 个工作
    if(L(t) == j){
        return false;
    }
    return true;
}
bound(t):

bound(t,j){
    swap(a[t], a[j]);
    if(cc + P[t][j] < bestCost){
        cc = cc + P[t][j];
        backtrack(t + 1);
        cc = cc - P[t][j];
    }
    swap(a[t], a[j]);
}

```

使用回溯法，迭代框架：

```

void backtrack(){
    int t = 1;
    while(t > 0){
        if(f(n,t) ≤ g(n,t))
            for(int i = f(n,t); i ≤ g(n,t); i++){
                a[t] = h[i];
                if(constraint(t)&&bound(t)){
                    if(solution(t)) output(a);
                    else t++;
                }
            }
        else t--;
    }
}

```

```

solution(t):
if(t == n)output(a);

```

(4) 此题的解空间树是排列数，时间复杂度是 $O(n!)$

某集团领导年底要到该集团的各个分部视察工作。已知该集团共有 n 个分部。给定 n 阶矩阵 C ，其中 $C[i][j]$ 表示领导从分部 i 到分部 j 所需要花费的代价。如果分部 i 到分部 j 不可达，则 $C[i][j] = +\infty$ 。求从分部 1 出发，视察每个分部一次后，再回到分部 1 所需的最少花费？若用优先队列式分支限界法求解该问题，回答以下问题：

(1) 每个搜索节点需要包含哪些信息，为什么？

(2) 给出部分解的下界函数，越大越好

(3) 给出部分解的上界函数，越小越好

(1) 包含：

$lcost$ ：子树的部分解下界

cc ：当前费用

$rcost$ ： $x[s:n-1]$ 中顶点最小出边费用和

s ：当前层，根节点到当前节点的路径为 $x[0:s]$

x ：需要进一步搜索的顶点是 $x[s+1][n-1]$

(2) 问题下界：将每个节点的最短路径求和。→每个顶点选择最短的两条边作为出入边，求和再除 2。

部分解下界：将已经经过的顶点的一条短的且没经过的边长换成已经经过的那条边。

部分解下界公式： $lb = (2 \sum_{i=1}^{k-1} C[r_i][r_{i+1}] + r_1 \text{行不在路径上的最小元素} + r_k \text{行不在路径上的最小元素} + \sum_{r_j \notin U} r_j \text{行最小的两个元素})$

(3) 问题上界：贪心算法，假设最后都能回到分部 1，从分部 1 开始，每次去最近的分部。

部分解上界：贪心算法，假设最后都能回到分部 1，从分部 i 开始，每次去最近的分部。

设某游艇俱乐部在长江沿岸设置了 n 个游艇出租站， $1, 2, \dots, n$ 。游客可在这些游艇出租站租用游艇，并在任何另一个游艇出租站归还游艇。已知游艇出租站 i 到游艇出租站 j 之间的租金为 $r(i, j)$ ， $1 \leq i < j \leq n$ 。若要计算从游艇出租站 1 到游艇出租站 n 所需的最少花费，完成以下问题。

(1) 若用动态规划方法求解该问题，给出问题解的递推定义。

(2) 填空完成以下动态规划算法程序。

```
/**
 * @param n 游艇租赁站数量
 * @param r 租赁站之间的租赁费用矩阵
 */
int minRent(int n, int ** r){
    for(int k = 2; k < n; k++){
        for(int i = 0; (1); i++){
            int j = i + k;
            for(int p = i + 1; (2); p++){
                int tmp = r[i][p] + r[p][j];
                if((3)) r[i][j] = tmp;
            }
        }
    }
    return (4);
}
```

$$(1) r[i][j] = \min \{ \min_{i < p < j} \{r[i][p] + r[p][j]\}, r[i][j] \}$$

(2)

(1) $i < n - k$

(2) $p < j$

(3) $tmp < r[i][j]$

(4) $r[0][n - 1]$

算法思想：

分治：将一个难以直接解决的大问题，分解成规模较小的相同子问题，直至这些子问题容易直接求解，并可利用这些子问题的解求出原问题的解。

动态规划：和分治法类似，将原问题分解成多个子问题，子问题之间往往不相互独立，保留已解决的问题的子问题，避免重复计算

贪心：总是找出当前最优解，局部最优解。

回溯、分支限界：从初始状态出发，搜索能达到是所有状态；当一条路走到尽头，则回退一步或多步，从另一个状态继续出发，直到所有路径都搜索过。

分支限界法：

装载问题（FIFO）：

```
Qnode{
    int weight;           // 集装箱重量
    Qnode parent;        // 父节点
    boolean Lchild;       // 是否是左儿子节点
}
```

装载问题（优先队列）：

```
Node{
    int uweight;          // 载重上界
    int level;            // 所在层
    Node parent;          // 父节点
    boolean leftCh;       // 是否为左儿子节点
}
```

最大团问题（优先队列）：

```
Node{
    int upperSize;        // 最大团节点数上界
    int level;            // 所在层
    Node parent;          // 父节点
    boolean leftCh;       // 是否为左儿子节点
}
```

批作业调度问题（优先队列）：

```
Node{
    int lb;               // 优先级信息
```

```

    int level;    // 所在层
    int[] x[];    // 当前作业调度顺序
    int sf2;      // 当前调度的完成时间和
    int f;        // 两个机器当前的完成时间
}

```

分支限界代码框架：

```

Q = {q0};    // Q 存储所有的活结点，初始化为根节点
void Branch&Bound(){
    while(Q! = ∅){
        select a node q from Q;    // 从Q中选择一个活结点
        Branch(q, Q1);    // 对 q 进行分支，产生Q1，分支时利用约束和界限进行剪枝
        add(Q1, Q);    // 将新产生的活结点加入Q
    }
}

```

装载问题：

```

int bestw = 0;    // 当前最优载重和
bestH[];    // 最优载重对应的解向量
Node beatNode;
MaxLoading(int[] w, int c, int n){
    Node cNode = null;    // 当前扩展节点
    int i = 1;    // 扩展节点的层
    int cw = 0;    // 扩展节点当前的载重
    int[] r = new int[n + 2];    // 定义剩余重量数组
    // 初始化
    r[n + 1] = 0;
    for(int j = n; j > 0; j--) r[j] = r[j + 1] + w[j];
    while( ){
        select;
        Branch( );
        add( );
    }
    while(bestNode! = null){    // 保留最优解
        bestH.add(bestNode.leftCh);
        bestNode = bestNode.parent;
    }
}

```

while :

while(*i*! = (*n* + 1)): // 如果搜索到叶子节点层, 且该节点的上界是所有节点的上界中最大的, 那么从其他节点出发产生的最大载重量都不可能比这个值大

select;

```
if(cw + w[i] ≤ c){           // 搜索左子树
    if(cw + w[i] > bestw){    // 更新 bestw
        // 更新 bestw, 构造新的 bestNode
        Branch()
    }
    add( ); // 将左节点加入队列
}
if(cw + r[i + 1] > bestw) add( ); // 利用上界剪枝
if(heap.size( ) == 0)break;      // 取下一个节点
cNode = heap.poll( );
i = cNode.level;
cw = cNode.uweight - r[i];
```

回溯法:

N 皇后问题 (递归):

```
void backtrack(int t){
    if(t > n) sum ++; // 解法+1
    else{
        for(int i = 0; i < n; i ++){
            x[t] = i;
            if(check(x, t)) backtrack(t + 1);
        }
    }
}
check(x, t) // 检查新放置的皇后是否和之前的皇后之间有冲突
```